

minio_setup

MinIO Setup — Helix OTA 1.0.0-MVP

Field	Value
Revision	2
Created	2026-06-07
Last modified	2026-06-07
Status	active
Status summary	Bootstrap procedure for the MinIO (S3-compatible) artifact blob store used by Helix OTA 1.0.0-MVP: bucket layout, a least-privilege service-account policy for ota-server, key issuance, and the byte-identity / range-GET requirements the artifact path imposes. Honors the LOCKED state-store split (PostgreSQL relational + MinIO/S3 blobs) and the artifact-path no-compression rule (ADR-0004). All access flows through the Storage catalogue brick.
Issues	HelixConstitution clause numbers UNVERIFIED. mc command flags target current MinIO client semantics and are UNVERIFIED against the exact pinned image. Whether the Storage brick supports range-GET for large artifacts is UNVERIFIED (potential upstream extend, submodule_reuse_map §5). In orchestrated (k8s) environments a managed/operator S3 may replace standalone MinIO (overview §6).
Fixed	Rev 2: clarified that helix in the mc commands is the local mc alias name (not a MinIO user) and that the actual root identity comes from the \$MINIO_ROOT_USER/ \$MINIO_ROOT_PASSWORD secret — no specific username is assumed.
Continuation	

Field	Value
	Pin the MinIO image + mc version and re-verify the commands; confirm Storage range-GET support and drop the UNVERIFIED tag; decide standalone-MinIO vs managed S3 for k8s; integrate the bucket/key bootstrap into the containers-substrate provisioning so it is reproducible, not manual.

Table of contents

1. Purpose and scope
2. Prerequisites
3. Bucket layout
4. Service-account policy (least privilege)
5. Key issuance and wiring
6. Artifact-path requirements (byte-identity + range GET)
7. Verification (four-layer)
8. Open / UNVERIFIED items
9. Sources

The table-of-contents requirement is mandated by HelixConstitution §11.4.61 (UNVERIFIED clause number). This document carries its ToC immediately after the metadata table.

1. Purpose and scope

MinIO is the **S3-compatible artifact blob store** for Helix OTA 1.0.0-MVP. It holds OTA artifacts — the AOSP `payload.bin`, its manifest, and the detached signature — while **PostgreSQL holds all relational state**. This split is LOCKED. [master §3; ADR-0003 §3]

This document covers the **one-time bootstrap**: creating the bucket, attaching a least-privilege policy, and issuing the access keys that ota-server consumes via the Storage brick. It does **not** cover MinIO HA/distributed-mode tuning (out of scope for MVP) and it does **not** put any direct S3 SDK call in Helix code — all access goes through the Storage catalogue brick. [submodule_reuse_map §3 Storage row] (UNVERIFIED: Storage provides an S3/MinIO backend.)

The MinIO service itself is defined in `docker-compose.mvp.yml`. In Kubernetes the MVP manifests intentionally omit MinIO; use a managed S3 or a MinIO operator (overview §6, UNVERIFIED choice).

2. Prerequisites

- MinIO running and healthy (compose: minio service healthcheck green).

- The MinIO client `mc` available (locally or `docker compose exec minio mc ...`).
- Root credentials supplied via container secrets (`MINIO_ROOT_USER_FILE` / `MINIO_ROOT_PASSWORD_FILE` in `docker-compose.mvp.yml`) — **never committed**; these root creds are used only for bootstrap, not by `ota-server`.

Register the MinIO endpoint as an mc alias (root creds used only for bootstrap).

```
mc alias set helix http://minio:9000 "$MINIO_ROOT_USER"
"$MINIO_ROOT_PASSWORD"
```

NOTE: `helix` here (and in every `mc ... helix ...` command below) is the local **mc alias name**, NOT a MinIO username. The actual root identity is supplied via `$MINIO_ROOT_USER/$MINIO_ROOT_PASSWORD` (from container secrets, `docker-compose.mvp.yml`); no specific root username is assumed. Likewise the issued service-account access key (§5) is generated by MinIO, not named `helix`.

(UNVERIFIED: exact `mc` flags against the pinned image; re-verify after pinning — §8.)

3. Bucket layout

A single bucket holds all artifacts; objects are keyed by release so listing is bounded.

Bucket	Purpose	Object key convention
helix-ota-artifacts	OTA artifacts	<release-id>/ payload.bin, <release-id>/ manifest.json, <release-id>/ payload.sig

```
mc mb helix/helix-ota-artifacts
```

Private by default – devices fetch via ota-server, never directly from MinIO.

```
mc anonymous set none helix/helix-ota-artifacts
```

Versioning ON so a release id never silently mutates (integrity defense-in-depth, ADR-0002).

```
mc version enable helix/helix-ota-artifacts
```

Devices **never** read MinIO directly; `ota-server` mediates every fetch (so auth, rate limit, and the artifact-path rules in §6 apply). The bucket stays private.

4. Service-account policy (least privilege)

`ota-server` gets a dedicated service account whose policy is scoped to the one bucket. Upload (release publishing) needs `PutObject`; serving needs `GetObject` (+ range); integrity checks need `GetObjectVersion`. No bucket-delete, no policy-admin.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["s3:ListBucket", "s3:GetBucketLocation"],
      "Resource": ["arn:aws:s3:::helix-ota-artifacts"]
    },
    {
      "Effect": "Allow",
      "Action": [
        "s3:PutObject",
        "s3:GetObject",
        "s3:GetObjectVersion"
      ],
      "Resource": ["arn:aws:s3:::helix-ota-artifacts/*"]
    }
  ]
}
```

```
mc admin policy create helix helix-ota-rw ./helix-ota-rw.json
```

Note there is **no s3:DeleteObject**: MVP releases are immutable; retirement is a relational-state flag in PostgreSQL, not a blob delete.

5. Key issuance and wiring

Issue a service account scoped to the policy above; its access/secret keys are the values ota-server consumes via the Storage brick.

```
mc admin user svcacct add helix "$MINIO_ROOT_USER" \
  --policy ./helix-ota-rw.json
# -> prints Access Key + Secret Key (store as container/
    orchestrator secrets, NOT in VCS)
```

Wire the issued keys into the deployment as **secrets** (overview §5):

- **Compose:** the minio_access_key / minio_secret_key Docker secrets in `docker-compose.mvp.yml` (file-mounted under `/run/secrets/...`).
- **Kubernetes:** the helix-ota-minio Secret (access-key / secret-key) referenced by `kubernetes/ota-server-deployment.yaml`.

The keys are referenced **by name/path only** in committed files; the literal values live outside version control. ota-server reads them through the Storage brick — no direct S3 SDK call elsewhere. [submodule_reuse_map §3 Storage row]

6. Artifact-path requirements (byte-identity + range GET)

These are hard requirements the store and its consumers must satisfy:

1. **Byte-identical serving.** `payload.bin` is the output of `update_engine's delta_generator` and is already internally compressed; it **MUST** be stored and served **byte-for-byte unchanged**. No transfer-encoding/content compression may be applied on the artifact path — re-compression yields ~no size win and **breaks the ZIP_STORED + byte-range contract** and the device-side `FILE_HASH/METADATA_HASH + AOSP` payload signature checks. [ADR-0004 §Decision artifact rule; aosp-update-engine §6, §7]
2. **Range GET.** Device-pull streaming uses HTTP Range requests; the store and the Storage brick must support range GET for large artifacts. (UNVERIFIED: Storage range-get support — potential upstream extend, `submodule_reuse_map` §5; ADR-0004 §1 range caveat.)
3. **Integrity at rest.** Bucket versioning is on (§3); ota-server verifies SHA-256 + the detached signature on **upload** (server-side defense-in-depth) before the object is considered published (MVP trust model). [ADR-0002 §4.1]

7. Verification (four-layer)

1. **Source-presence gate.** This doc exists with metadata table + ToC; the policy JSON and bucket name match the references in `docker-compose.mvp.yml` and `kubernetes/ota-server-deployment.yaml`; no literal key value appears in any committed file.
2. **Artifact gate.** The policy JSON parses; the mc command set is syntactically valid; the bucket name is a single source of truth across compose/k8s/this doc.
3. **Runtime / integration gate.** After bootstrap: `mc ls helix/helix-ota-artifacts` succeeds with the service-account key; ota-server PUTs a signed test artifact, a range-GET returns **byte-identical** bytes, and server-side SHA-256 + signature verify passes; an anonymous direct GET against MinIO is **denied** (bucket private).
4. **Mutation meta-test.** Mutating the setup must be caught: granting `s3:DeleteObject` → immutability test fails; setting the bucket public → anonymous-GET-denied test fails; enabling compression on the artifact path → byte-identity test fails (§6.1). A mutation no test catches is a missing test (mutation immunity). [master §13; §1.1]

8. Open / UNVERIFIED items

1. **HelixConstitution clause numbers** — UNVERIFIED (corpus convention).
2. **mc command/flag accuracy** against the pinned MinIO image — UNVERIFIED until pinned.
3. **Storage brick range-GET + S3/MinIO backend** — UNVERIFIED (`submodule_reuse_map` §3, §5).
4. **k8s object store** — standalone MinIO vs managed S3 / MinIO operator — UNVERIFIED (overview §6); MVP k8s manifests omit MinIO.
5. **Image digest** for MinIO — must be pinned before deploy — UNVERIFIED (overview §8).

9. Sources

- [overview.md](#) — deployment overview (service set §3.3, secrets §5, environments §6).
- [docker-compose.mvp.yml](#) — minio service + secrets wiring.
- [kubernetes/ota-server-deployment.yaml](#) — S3 env + helix-ota-minio Secret reference.
- [../../../../research/adr/adr-0002-supply-chain-trust.md](#) — MVP trust model (signing + SHA-256 + AVB; server-side verify on upload).
- [../../../../research/adr/adr-0004-transport.md](#) — artifact-path no-compression / byte-identity / range-request rule.
- [../../../../00-master/submodule_reuse_map.md](#) — Storage brick binding; range-get upstream-extend note (§3, §5).